

Pixel Art Is a Palette, Not a Downscale: Rebuilding Targets and Decoding for Tiny Sprite Generation

Anonymous CVPR submission

Paper ID *****

Abstract

001 *Text-to-image models fail at the smallest images people draw*
002 *on purpose: 12–24 px RGBA game sprites, where a hand-*
003 *ful of colours and hard alpha edges are the medium. We*
004 *show that the usual failure is not a modelling failure but*
005 *a target failure. In a sprite corpus assembled the usual*
006 *way — cut sheets into components, bucket by size, down-*
007 *scale to fit — only 7.1% of the images are natively 16px*
008 *or smaller; the rest is 25–64 px artwork box-downscaled*
009 *into the low-resolution buckets, and 4,888 pixel-identical*
010 *duplicates leave a twin in training for 16.2% of the held-out*
011 *set. A model fitted to those targets learns soft, fifty-colour*
012 *blobs, and the evaluation reference, drawn from the same*
013 *pool, rewards exactly that. We rebuild the targets and eval-*
014 *uate against references that contain only real low-resolution*
015 *pixel art. Under this protocol the data work alone takes*
016 *FD-DINOv2 from 167.2 to 117.9 at 16px. We then close*
017 *the remaining gap where it actually lies: real sprites use*
018 *a median of six opaque colours while every sampling rule*
019 *we tested emits more than fifty. Projecting the predicted $\hat{x}_0$*
020 *onto a per-sprite k -colour palette during the late denoising*
021 *steps reaches 31.8 ± 0.5 — better than quantising finished*
022 *samples (41.6), than one corpus-wide palette (120.1) and*
023 *than a median filter that removes the same dither (112.2).*
024 *The rule is a property of the recipe rather than of the net-*
025 *work: a transformer backbone that trails the UNet by 36.5*
026 *FD without it lands within 0.7 with it. Applied to sprites*
027 *from generators we did not train, it improves native FD by*
028 *36–41% while preserving 80% of the silhouette. Against ex-*
029 *ternal systems given the same palette post-process we reach*
030 *53.1 versus 134.4 for gpt-image-2 with downscaling.*

031 1. Introduction

032 Game sprites are among the smallest images anyone draws
033 on purpose: an item or a character on a 12–24 px RGBA can-
034 vas, built from hard edges, a handful of flat colour regions
035 and a transparent background. At these sizes there is no tex-

ture in which to hide a mistake — one wrong pixel changes
the silhouette, one blended colour reads as blur. The obvious
recipe, prompting a large text-to-image model and shrinking
its output, produces something recognisable but not a sprite:
it keeps the soft shading and the hundred-odd colours of an
illustration (Figure 1).

This paper is about a failure one level below the model.
We train a dedicated generator for this medium — a 72.5M
pixel-space diffusion model over RGBA sprites, conditioned
on a frozen CLIP text encoder and a resolution-bucket label
— and find its weaknesses inherited from what it is
asked to imitate. We assembled its corpus the way such cor-
pora are assembled: cut sprite sheets into connected com-
ponents, label each sprite with a resolution bucket, down-
scale to fill the bucket asked for, and split train from test
by file path. Every defect below follows from that recipe
rather than from one dataset. Only 7.1% of the images are
natively 16 px or smaller; two thirds are 25–64 px artwork
the pipeline box-downsamples into the low-resolution buck-
ets; and 4,888 pixel-identical duplicates leave a twin in train-
ing for 16.2% of the held-out set, because a path-based split
cannot see that two files are the same image. A model fitted
to those targets learns what they are: soft, fifty-colour blobs.
Worse, the standard evaluation reference is drawn from the
same pool, so it rewards the blur it caused, and the ranking
of methods under it does not survive contact with real pixel
art.

We change both the targets and the yardstick. On the
data side we remove the duplicates, forbid feeding a sprite
to a bucket more than 1.5× below its native size, and add
14,999 natively small sprites from permissively licensed col-
lections, captioned with a vision–language model. On the
evaluation side we define the *native protocol*: FD-DINOv2
against real sprites that are natively at most R px, never
downscaled. At 16 px the data work alone moves the model
from 167.2 to 117.9.

What remains is a property of the medium that continu-
ous diffusion cannot express. Real 16 px sprites use a me-
dian of six opaque colours and 46% of their adjacent opaque
pixel pairs are exactly equal; every sampling rule we tested



Figure 1. Four rows of 16 px RGBA sprites, nearest-neighbour magnified, with the median number of opaque colours per row. Real sprites drawn at this size use six; the corpus’s own 16 px targets, which are 25–64 px artwork box-downscaled, use 49; gpt-image-2 rendered large and reduced uses 111; ours uses four. The third row is the easiest on the eye and the furthest from the medium.

emits more than fifty colours and reaches 8%, spending the difference on few-level dither inside regions that should be flat. We close that gap inside the sampler: during the late denoising steps we project the predicted $\hat{x}_0$ onto a per-sprite k -colour palette (batched k -means over the opaque pixels, alpha untouched) and continue, so the remaining steps repair the seams hard quantisation leaves. At 16 px this takes FD from 87.1 to 31.8 ± 0.5 , where quantising the finished samples reaches only 41.6.

Two results say the rule is not an artefact of our network. Replacing the UNet with a 69.7M transformer, everything else held fixed, gives a model that trails by 36.5 FD under plain guidance and by 0.7 once the projection is on: the decoding rule absorbs the difference between the two architectures. And the rule applies to sprites we did not generate — re-noising the output of gpt-image-2, FLUX.2 with a pixel-art LoRA, or SDXL with Pixel Art XL and denoising it under our recipe improves native FD by 36–41% at every resolution while keeping about 80% of the input silhouette.

We also report a negative result we consider as informative as the positive one. Guiding a bucketed model away from its own lower-resolution prediction, and a one-pass variant that internalises the rule into the weights, are the best samplers we have under the downscale-biased reference and among the worst under the native one. Which reference set one scores against decides which of these conclusions one publishes.

Contributions.

- A diagnosis of the targets used for tiny-sprite generation — native-resolution statistics, duplicate leakage, and the resulting bias of the standard reference — and a rebuilt corpus plus the native protocol that removes both.
- **Palette-projected sampling**, which lowers FD-DINOv2 at 16 px from 87.1 to 31.8 ± 0.5 over three seeds, beats post-hoc quantisation (41.6) and two controls that isolate what it is doing, and costs 0.8 points of caption retrieval at $k=6$, where the samples match the colour statistics of real sprites.
- Evidence that the rule belongs to the recipe rather than the network: it absorbs 98% of the gap between two architectures, and it improves the sprites of three systems we did not train.
- An external comparison in which every system receives the same palette post-process: at 16 px we reach 53.1 ± 2.3 against 134.4 for gpt-image-2 with downscaling, 150.1 for FLUX.2 with a pixel-art LoRA and 244.2 for SDXL.

2. The medium and the protocol

What the corpus actually contains. Our starting corpus is 37,489 unique RGBA sprites cut from OpenGameArt sheets by connected components and filtered, assembled before we knew what was wrong with it and kept here as the object of study. Its low-resolution buckets are not made of low-resolution art. Only 7.1% of the sprites are natively 16 px or smaller and 33% are natively 24 px or smaller; two thirds are 25–64 px artwork that the training pipeline box-

132	downsamples into whatever bucket it is asked for. The con-	182
133	sequences are measurable in the targets themselves: a real	183
134	sprite drawn at 16 px has a median of six opaque colours	
135	and 46% of its adjacent opaque pixel pairs are exactly equal,	
136	while the 25–64 px artwork that feeds the same bucket ren-	
137	ders to 49 colours and a fifth of that (Figure 1, rows 1–2). A	
138	random sample of the whole corpus sits between them at 34	
139	colours, since 7.1% of it really is small.	
140	Duplication compounds it. Hashing every sprite by its	
141	exact pixel content finds 4,888 extra copies, and because a	
142	hold-out set defined by file path cannot see that two paths	
143	hold the same image, a twin of a held-out sprite stays in	
144	training: 16.2% of the 3,000 evaluation sprites have one,	
145	counting a twin anywhere training is allowed to see. We	
146	drop every extra copy and every copy of a held-out sprite.	
147	Rebuilt targets. Three changes: remove the duplicates;	
148	forbid feeding a sprite to a bucket more than 1.5× below	
149	its native size, so a 64 px illustration can no longer become	
150	a 16 px target; and add 14,999 natively small sprites cut	
151	from three permissively licensed OpenGameArt collections	
152	(CC-BY-4.0, CC-BY-SA-3.0, OGA-BY-3.0), captioned with	
153	a vision–language model and filtered for fonts, UI elements	
154	and near-identical animation frames. Per-file licence records	
155	accompany the added material.	
156	The native protocol. The reference set decides the result,	
157	so we define it explicitly. <i>Native</i> reference: all real sprites	
158	that are natively at most R px, never downsampled — 2,675 of	
159	them at 16 px, split in half so a real-versus-real floor can be	
160	computed on a disjoint set (4.47 at 16 px). <i>Biased</i> reference:	
161	what one gets by default, a random 3,000 of the corpus, of	
162	which 92.9% is downsampled artwork at 16 px (floor 3.45).	
163	We report both throughout, because the two rank methods	
164	differently and the disagreement is part of the result.	
165	Scores are Fréchet distances of DINOv2-small CLS fea-	
166	tures [17, 22] computed at the native resolution: sprites are	
167	composited on white and nearest-neighbour upsampled to	
168	224 px. One sample per caption for 3,000 held-out cap-	
169	tions, which are recaptions of the held-out sprites, so the	
170	prompt distribution matches the data. Seed-to-seed stan-	
171	dard deviation of our headline configuration is 0.49 over	
172	three seeds, and an independent retraining of the same recipe	
173	lands within 2.5 FD, so we do not read differences below	
174	that scale. We also report CLIP ViT-B/32 image–text simi-	
175	larity and retrieval of the own caption among 100, with the	
176	distractor draw re-seeded per set.	
177	Leakage control. Exact duplicates are removed, but near-	
178	duplicates are not: by an 8×8 luminance-plus-alpha aver-	
179	age hash, 1,241 of the 3,000 held-out prompts (41.4%) still	
180	have a twin in training. Rescoring every configuration on	
181	the 1,759 prompts that do not moves each number by at most	
	3.0 FD — the largest single change is 167.2 to 164.2 — and	
	changes no ordering, so the main tables use the full set.	
	3. Related work	
	Pixel art. Most work on pixel art converts an existing im-	
	age rather than generating one: joint palette-and-pixel opti-	
	misation [7], contrast-aware downscaling [26], learned pix-	
	elization [9, 25], and optimisation-based stylisation with	
	a diffusion prior [4]. Text-to-sprite generation in practice	
	is served by community LoRAs on large models [15, 16],	
	which have no published description of their data or train-	
	ing. Palette-conditioned generation has been proposed [14],	
	with the palette supplied as an input; we estimate it per sprite	
	from the model’s own prediction and need no extra condi-	
	tioning.	
	Guidance. CFG [10] extrapolates away from an uncondi-	
	tional prediction; a long line of work reshapes that update	
	[6, 13, 19, 21] or replaces the reference with a weaker pre-	
	diction from the same network — a smaller model [12], per-	
	turbed attention [1, 11], or a random condition [20]. We	
	use independent-condition guidance, which needs no uncon-	
	ditional training, and show it is the rule that survives our	
	change of reference set.	
	Constraints inside the sampler. Projecting the predicted	
	$\hat{x}_0$ at every step is the standard way to impose a hard con-	
	straint on a diffusion model [5, 23]; discrete-state formula-	
	tions impose quantisation by construction instead [2, 8]. Our	
	projection is that device applied to the property that defines	
	this medium, and it changes nothing about training.	
	Reference sets decide rankings. That a benchmark’s ref-	
	erence set can decide an ordering is known in generative	
	evaluation, from FID’s sensitivity to the feature extractor	
	and preprocessing [18, 22] to duplicate contamination in	
	image corpora [3, 24]. Our instance is concrete: for tiny	
	sprites a corpus-sample reference is 92.9% downsampled art-	
	work, and that single fact reverses the ranking of the guid-	
	ance methods we would otherwise report.	
	4. Palette-projected sampling	
	The model. A pixel-space diffusion model over RGBA	
	sprites: a 72.5M UNet with cross-attention on a frozen CLIP	
	ViT-B/32 text encoder and a learned embedding of the <i>reso-</i>	
	<i>lution bucket</i> $b_R \in \{12, 16, 20, 24, 32, 48, 64\}$, trained with	
	the usual ϵ -objective on a squared-cosine schedule. A sprite	
	is centred on an $R \times R$ canvas with hard alpha. Nothing here	
	is novel; it is the smallest model that lets us study the targets	
	and the decoding rule.	



Figure 2. Palette-projected sampling at 16 px, same prompts and seed. Without projection (top) flat regions carry a few levels of dither; projecting the predicted $\hat{x}_0$ onto four or six colours during the late steps (middle, bottom) removes it and the remaining steps repair the seams. The three rows correspond to rows of Table 1; we give the numbers there rather than in this caption, since the panels were rendered at two different guidance weights.

The rule. A 16 px sprite is defined as much by how few colours it uses as by its shape, and a continuous denoiser in RGB has no mechanism that snaps a region to one value. At every step with $t/T \leq f$ we form the current estimate $\hat{x}_0 = (x_t - \sqrt{1 - \bar{\alpha}_t} \tilde{e}) / \sqrt{\bar{\alpha}_t}$, cluster its *opaque* RGB pixels into k groups with a batched k -means (8 iterations, initialised at evenly spaced luminance quantiles), replace each pixel by its centroid, leave the alpha channel and transparent pixels untouched, and map the result back to a noise prediction:

$$\hat{x}_0^{\text{pal}} = \Pi_k(\hat{x}_0), \quad \tilde{e} \leftarrow (x_t - \sqrt{\bar{\alpha}_t} \hat{x}_0^{\text{pal}}) / \sqrt{1 - \bar{\alpha}_t}, \quad (1)$$

before taking the scheduler step. The projection is a hard constraint on the model’s own belief rather than a post-process: the steps that follow see the quantised state and repair the seams it creates, which is why doing it inside the sampler beats quantising the finished image (31.8 against 41.6 at 16 px). It adds no network evaluations; k -means on at most 576 pixels is free next to a UNet pass. We use $f = 0.5$; the optimum is flat over $[0.4, 0.5]$ and running the projection for the whole schedule costs two points, so the timing matters far less than the projection.

5. Experiments

Main result. Table 1 separates the three things that changed. Rebuilding the targets moves the model from 167.2 to 117.9 under the native protocol without touching the sampler. Independent-condition guidance takes it to 87.1, and palette projection to 31.8 ± 0.5 over three seeds — a step of 55.3, against 30.8 for moving from plain guidance

Table 1. 16 px, 3,000 held-out captions. *Native*: reference of sprites natively ≤ 16 px (floor 4.47). *Biased*: the conventional corpus-sample reference (floor 3.45), 92.9% of it downsampled artwork. The two columns rank the same methods in opposite orders.

Targets	Sampler	Native ↓	Biased
original	CFG 1.5 (best)	167.2	11.1
policy fixed	CFG 1.5	123.1	14.8
+ new sprites	CFG 1.5	117.9	13.6
+ new sprites	cross-res. ref., $w=1.5$	114.9	14.2
+ new sprites	internalised, 1 pass	170.3	10.4
+ new sprites	ICG $w=2$ [20]	87.1	20.7
+ new sprites	ICG + post-hoc 4-colour	41.6	111.0
+ new sprites	ICG + projection, $k=6$	36.9	71.1
+ new sprites	ICG + projection, $k=4$	31.8 ± 0.5	111.2

to the best guidance rule. Projecting inside the sampler is what makes that step work: quantising the finished samples to the same four colours reaches only 41.6, because nothing repairs the seams quantisation introduces. The two columns disagree wherever the sampler changes: every guidance and projection row that improves the native score makes the biased one worse. The data changes are the exception — adding natively small sprites improves both — which is what one would expect of a change that makes the targets more like pixel art without changing what the sampler does. Section 7 takes the disagreement apart.

Is it the palette, or just fewer colours? Two controls replace Π_k at exactly the same steps. One projects onto a *single* four-colour palette shared by every sprite, learned by k -means over the reference set: the same colour count without the per-sprite structure. The other applies a 3×3 median filter, which removes the same dither and imposes no palette. Both are *worse* than not projecting at all — 120.1 and 112.2 against 87.1 — so the gain is neither “fewer colours” nor “smoother”, but each sprite’s own small palette.

Palette size, with a validation split. Sweeping k at 16 px gives 38.1 / 31.8 / 33.7 / 36.9 / 50.6 for $k = 3/4/5/6/8$, and 39.9 when k is drawn per sprite from the corpus histogram. Because that sweep and the reported number share a split, we also split the captions in half by index: choosing on one half selects the same $k=4$, which scores 33.36 on the disjoint half against 38.63 / 35.62 / 38.27 / 43.76 for the alternatives (values at $n=1500$ are not comparable to $n=3000$ ones, as FD is biased by sample size). The FD optimum $k=4$ is nonetheless *below* the real median palette: measured on the samples, $k=6$ is the setting that reproduces the medium — six opaque colours and 43% exactly-equal neighbours, against six and 46% for real sprites — while $k=4$ gives four colours and 54%, flatter than the real thing. We report $k=4$

as the best-FD setting and $k=6$ as the one to use.

Prompt alignment. Projection trades a little text faithfulness for a large distributional gain. CLIP 100 cos / retrieval@1 among 100 are 27.60 / 22.8% for ICG alone, 27.38 / 22.0% at $k=6$ and 27.07 / 19.2% at $k=4$, against 27.66 / 22.6% for the real held-out sprites the captions were written from: at $k=6$ our samples are as retrievable as real sprites. The cost is smaller on a larger canvas but stops shrinking — 3.6 points at 16 px, 1.9 at both 20 and 24 px.

Is it the network? Every number above comes from one convolutional network. We retrained the 20k-step pilot with the UNet replaced by a 69.7M-parameter transformer (adaptive-norm DiT; the bucket enters through the text context, as that design has no class-conditioning path), leaving corpus, schedule, sampler and evaluation untouched. At 20k steps the transformer is the weaker model by a wide margin — 185.6 against 149.1 native FD under plain CFG — and yet under the reported configuration the two land at 42.4 and 43.1 (two seeds each, spread 0.3 and 0.4). The projection absorbs 98% of the difference between the two architectures, so it is a property of the recipe and not of convolutions.

More data, and why a short pilot cannot tell you whether it helps. The corpus is only 7.1% natively small, so more natively small sprites is the obvious remedy. We found two sources: the Kenney CC0 packs (6,566 usable rows, a median palette of three colours, but only 41.8% whole objects) and the OpenGameArt CC-BY-3.0 bundle our rebuild had never touched (11,988 usable rows against which exact pixel hashing finds 13 duplicates, 82.4% whole objects), together taking training from 86,290 rows to 118,378. At 20k steps the addition looks decisive: the reported configuration moves from 54.2 to 53.2 with the Kenney packs and to 42.9 with the CC-BY-3.0 bundle, a 19.4% gain and twenty times the seed spread. Trained to the same 80k steps as every other model here, the enlarged corpus is *worse*: 39.3 ± 0.3 against 31.8 ± 0.4 over three seeds each, with plain CFG unchanged. The ordering at 20k is the reverse of the ordering at 80k, and only the second is the model anyone would ship. We offer no mechanism, because the obvious one does not survive measurement — the natively-small fraction of the added material is 24.0% against 28.6% for what was already there, nowhere near enough to explain a 24% regression.

This is the second time in this work that a 20k-step comparison inverted by 80k; the first was capacity, where a 2.2× larger network led at 20k (46.1 against 54.2), had lost by 40k (38.3 against 35.8) and was clearly behind at 80k (38.5 against 31.8). Short pilots are how this kind of study is made affordable, and in this setting they are safe for rejecting a change and not for accepting one. Every headline number here is measured at full length for that reason. The reversal

Table 2. External comparison, 200 held-out captions, native protocol. Every row is scored after the same k -colour post-process ($k = 4/8/6$ at 16/20/24 px); ours applies it inside the sampler instead. “Real” are held-out sprites of the original, mostly down-scaled corpus.

System	16 px	20 px	24 px
Ours	53.1 ± 2.3	152.5	192.2
Real sprites (mixed provenance)	77.0	165.5	150.7
gpt-image-2 + downscale	134.4	448.6	478.0
FLUX.2-klein + pixel-art LoRA	150.1	473.9	472.4
SDXL + Pixel Art XL	244.2	526.3	527.2

is not an artefact of the metric: the provenance classifier of Table 5, which is asked a different question, reproduces it — at 20k steps it calls 74.4% of the enlarged corpus’s samples natively drawn against 69.0% for the base corpus, and at 80k steps 70.2% against 73.5%.

Other resolutions. The recipe transfers, though the best palette size does not track the canvas: native FD with ICG $w=2$, three seeds at the best k of each resolution, is 143.2 ± 0.8 at 12 px ($k=8$), 31.8 ± 0.5 at 16 px ($k=4$), 96.5 ± 0.9 at 20 px ($k=8$) and 123.6 ± 0.5 at 24 px ($k=6$). The smallest canvas wants the largest palette, which we cannot explain and do not dress up: at 12 px a four-colour projection has so few pixels to work with that it removes structure rather than dither. The absolute values are not comparable across resolutions — each has its own reference set and floor.

External systems. Table 2 compares three publicly available pipelines on 200 held-out captions, each generating at its native size and reduced to the sprite canvas by the same code we use for real data, with the same palette post-process. Our row is the mean of three seeds at 16 px (50.8 / 52.9 / 55.4); the 20 and 24 px columns are single seeds, and on 200 prompts a seed is worth about 2 FD, so they should be read with that. Without the post-process the external systems score 284.4 / 318.9 / 436.4 at 16 px, so the palette step helps them too; they remain 2.5–4.6× further from real pixel art than our model at every resolution. At 16 and 20 px we are also closer to native pixel art than the mixed-provenance sprites a corpus-sample protocol calls ground truth, which is a statement about that corpus rather than about our model. At 24 px the real sprites are still ahead of us.

6. The rule on other systems

Quantising a finished external sample helps but cannot repair the seams quantisation leaves, the same limit we measured on our own samples. The projection can nevertheless be applied to another system’s output: re-noise its sprite to



Figure 3. Eight captions drawn at random. Rows: gpt-image-2 rendered large and reduced with the same palette post-process as every other row; the same sprite re-noised to σ of the schedule and denoised by our model; our model generating the caption from noise; and the held-out sprite. The refined sprites keep the subject and also look better than our own samples for the same captions, which score 86.5 and 53.1 native FD respectively: what our model still lacks is not the decoding rule but the ability to draw the subject.

Table 3. The decoding rule applied to external sprites, 200 captions, 16 px. σ is the fraction of the schedule re-run. Retention is measured against the system’s own output; the floor is our model generating the same captions from noise, i.e. what “no retention” looks like.

System	Native FD $\downarrow$ at σ			Ret. @ 0.6 IoU
	post-hoc	0.4	0.6	
gpt-image-2	134.4	115.9	86.5	0.788
FLUX.2 + LoRA	150.1	116.1	88.2	0.803
SDXL + Pixel Art XL	244.2	197.2	155.0	0.813
Ours, from noise	53.1 $\pm$ 2.3			0.513

a fraction σ of the schedule and denoise it under our configuration. Table 3 reports both halves — how much closer to real pixel art the result gets, and how much of the original survives — because at $\sigma=1$ the procedure is simply our own model, and a distributional score alone cannot tell repair from replacement. Retention is measured in pixel space, as the DINOv2 cosine is saturated at this size: two unrelated sprites for the same caption already score 0.918.

At $\sigma=0.6$ about 80% of the silhouette survives, well

above the 0.46–0.53 floor of a freshly generated sprite, and native FD improves by 36–41% for all three systems; the same holds at 20 and 24 px (448.6 to 265.7 and 478.0 to 299.0). Sweeping σ on gpt-image-2 gives a monotone trade-off with no knee, and the endpoint is a self-check: at $\sigma=1$ the run reproduces our own samples on all three measurements — FD 53.6 against 53.1 $\pm$ 2.3, retention 0.518 against 0.513, retrieval 16.0% against 15.0%. Caption retrieval falls from 22.0% for gpt-image-2’s own sprite to 18.5% after refinement and 15.0% when our model draws the caption itself, so refinement keeps most of a large generator’s semantic advantage while moving the style towards the medium. It does not close the gap to generating natively, and the gap widens with resolution: a post-process fixes how a sprite is drawn, and the larger the canvas the more of the remaining distance is structure it cannot touch.

The provenance classifier of Table 5 agrees, having been asked nothing about FD: it calls 33.0% of gpt-image-2’s post-processed sprites natively drawn, and 40.5 / 50.0 / 53.0 / 61.5 / 70.0% of the same sprites refined at $\sigma = 0.2$ to 1. FLUX.2 rises from 30.5% to 58.5% at $\sigma=0.6$. The endpoint again lands on our own samples (70.0% against 67.5%), which is the same self-check in a second instrument.

Table 4. Per-image statistics at 16 px: median opaque colours and the fraction of adjacent opaque pixel pairs with identical RGB. The two reference sets are different populations, and each sampler sits next to the one it scores well against.

	Colours	Flat pairs
Native reference (real, drawn at 16 px)	6	0.46
Biased reference (corpus sample)	34	0.20
CFG 1.5	52	0.08
ICG $w=2$	49	0.08
Ours, projection $k=6$	6	0.43
Ours, projection $k=4$	4	0.54
gpt-image-2, reduced	93	0.01

7. Discussion and limitations

Why the two protocols disagree. Table 4 makes the disagreement concrete. The biased reference is 92.9% down-scaled artwork — 34 colours and a fifth of its neighbouring pixels equal, against six colours and nearly half for real sprites — so it is a reference for *blurred illustrations*, and the sampler that scores best against it is the one that produces them. That is why heavy guidance wins there and loses here; why the internalised cross-resolution rule — which makes samples look like a lower-resolution version of themselves, i.e. like downscaled art — is the best row of the biased column (10.4 against 13.6 for the best CFG) and the worst of the native one (170.3 against 117.9); and why palette projection, which moves samples from 52 colours to four, improves the native score by a factor of 2.7 while making the biased score five times worse. Neither number is wrong; they measure distance to different populations, and only one of those populations is pixel art.

Where the remaining error lives. The projection fixes the colour statistics and leaves one visible difference untouched. Real sprites drawn at 16 px cover 26% of their canvas; the corpus-sample reference covers 37%, because artwork reduced to fit a frame fills it; and our samples cover 34% with the projection and 35% without it. On the property the decoding rule does not touch, our model sits beside the reference we argue is wrong, not beside the one we argue is right — and the enlarged corpus, which is worse on every other measure, is further still at 38%.

The provenance classifier picks the same thing out without being told to look for it: of our 3,000 samples, the 834 it declines to call natively drawn cover 45% of the canvas against 34% for the 2,166 it accepts, while their palettes are indistinguishable (3.9 against 3.8 colours). What is left of the gap to real pixel art is not colour and not dither; it is that our sprites are drawn too large for the frame, with too little of the transparent margin that a sprite authored at this size is surrounded by. The mechanism is in the bucketing. A

sprite natively smaller than its bucket is centred and keeps a margin: the natively-16 px group covers 26% of the canvas after preprocessing. A sprite larger than its bucket is resized to fill the frame: the 17–24 px group, which our 1.5 $\times$ policy admits, covers 40%. Four in five sprites in the 16 px bucket come from the second group, so the bucket averages about 37% and the model reproduces it at 34%. Even a policy that forbids more than 1.5 $\times$ downscaling still teaches the model to fill the frame. That is a property of the targets — the same contamination this paper is about, surviving one level down — and not something a sampler can repair; the fix is to preserve a sprite’s native margin rather than fit it to the canvas, which we have not tried.

A second question put to the same features. Whether an image was authored at sprite resolution is a fact about the file, not a matter of taste, so it can be learned. We fit a logistic regression on DINOv2 features to separate real sprites drawn at 16 px from 25–64 px artwork reduced to 16 px, holding out the half of the native pool used everywhere else in this paper, and ask it what fraction of each set it calls natively drawn. It reaches 93.0% accuracy on its own held-out split and calls 94.9% of the unseen real sprites native, so unlike the judge below it passes its control before being asked anything.

Table 5. Fraction of each set a provenance classifier calls natively drawn. It is trained only on real sprites, on the question of how they were authored, and never sees a generated image during fitting.

Set	Called natively drawn
Real held-out sprites (control)	94.9%
Ours, projection $k=4$	67.5%
Ours, projection $k=6$	63.0%
Ours, no projection	36.8%
gpt-image-2, palette post-process	33.0%
FLUX.2 + LoRA, palette post-process	30.5%
gpt-image-2, reduced only	4.5%

The projection nearly doubles the fraction, from 36.8% to 67.5%, and the external systems stay near a third even with the same palette post-process; reduced without it, gpt-image-2 is called natively drawn 4.5% of the time. The table also says plainly how far we still are: 67.5% against 94.9% for real sprites. Fitting the same probe at the other resolutions gives the same shape — 42.4% to 58.7% at 20 px and 28.1% to 42.7% at 24 px, with 91.5% and 91.6% of unseen real sprites called native — although the classifier itself gets weaker as the canvas grows (93.0%, 90.3%, 87.7% held-out accuracy), which is what one would expect when authored and downscaled sprites have more pixels in which to resemble each other.

This is a different question, not an independent second

opinion, and we would rather say so than let it be read as one. It uses the same features as the FD, and a classifier separating these two populations can be expected to key on palette size and edge hardness — which is exactly what our method changes. It therefore supports the claim that our samples match the statistics of natively drawn pixel art, and supports nothing about whether they are good sprites. Where it and the FD disagree, that limit is visible: at 20k steps the classifier calls 79.6% of the transformer’s samples natively drawn against 74.4% for the UNet’s, while their FD under the same configuration is within half a point. The classifier sees the palette; the FD also sees what was drawn.

Can a model be asked instead of people? We put the same pairs to a vision–language model: 560 forced choices over 40 captions, with the real held-out sprites in the pool as a control. The control fails. Asked which sprite looks more like real low-resolution pixel art, the judge puts FLUX.2 and gpt-image-2 above the real sprites themselves (72% and 70% head to head); replacing the question with the concrete distinction — drawn pixel by pixel at this size, or a larger picture shrunk down — moves the real sprites from 40.6% to 48.8% and the blurriest baseline from 41.2% to 27.5%, but FLUX.2 still beats the real sprites 68% of the time. Our own samples tie with real sprites at 50% under both wordings. We report this as a negative result about the instrument: whether an image was authored at sprite resolution is not something a general-purpose vision model recovers from the image, which is the work the native reference set is doing.

Limitations. *Scope.* Everything is measured on models we trained, on one family of sprite corpora; the transformer run rules out convolutions as the explanation but not a shared bias of pixel-space diffusion, and we have not tested a second domain. *Metric.* FD with DINOv2 on 12–24 px images upsampled to 224 px is sensitive to exactly the colour and contrast statistics the projection changes. Inception clean-FID on the same samples and reference preserves the ordering, including both controls, but compresses it — a 2.8× gap becomes 1.10× — because an encoder that resizes to 299 px cannot see hard edges at 16 px. We have no human study, and it remains the obvious next check. *Tuning.* k transfers across a validation split; for f the honest statement is that the optimum is flat over $[0.4, 0.5]$ rather than validated. *Where we still lose.* At 24 px real sprites remain closer to native pixel art than our samples, and a large generator plus our post-process still draws subjects better than we generate them.

References

- [1] Donghoon Ahn, Hyoungwon Cho, Jaewon Min, Wooseok Jang, Jungwoo Kim, SeonHwa Kim, Hyun Hee Park, Ky-

- ong Hwan Jin, and Seungryong Kim. Self-rectifying diffusion sampling with perturbed-attention guidance. In *European Conference on Computer Vision (ECCV)*, 2024. 3
- [2] Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. Structured denoising diffusion models in discrete state-spaces. In *Advances in Neural Information Processing Systems*, 2021. 3
- [3] Björn Barz and Joachim Denzler. Do we train on test data? Purging CIFAR of near-duplicates. In *Journal of Imaging*, 2020. 3
- [4] Alexandre Binniger and Olga Sorkine-Hornung. SD- π XL: Generating low-resolution quantized imagery via score distillation. In *SIGGRAPH Asia 2024 Conference Papers*, 2024. 3
- [5] Hyungjin Chung, Jeongsol Kim, Michael T. Mccann, Marc L. Klasky, and Jong Chul Ye. Diffusion posterior sampling for general noisy inverse problems. In *International Conference on Learning Representations*, 2023. 3
- [6] Hyungjin Chung, Jeongsol Kim, Geon Yeong Park, Hyelin Nam, and Jong Chul Ye. CFG++: Manifold-constrained classifier free guidance for diffusion models. In *International Conference on Learning Representations (ICLR)*, 2025. 3
- [7] Timothy Gerstner, Doug DeCarlo, Marc Alexa, Adam Finkelstein, Yotam Gingold, and Andrew Nealen. Pixelated image abstraction. In *Proceedings of the 10th International Symposium on Non-Photorealistic Animation and Rendering (NPAR)*, pages 29–36. Eurographics Association, 2012. Extended version: *Computers & Graphics* 37(5):333–347, 2013, doi:10.1016/j.cag.2012.12.007. 3
- [8] Shuyang Gu, Dong Chen, Jianmin Bao, Fang Wen, Bo Zhang, Dongdong Chen, Lu Yuan, and Baining Guo. Vector quantized diffusion model for text-to-image synthesis. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022. 3
- [9] Chu Han, Qiang Wen, Shengfeng He, Qianshu Zhu, Yinjie Tan, Guoqiang Han, and Tien-Tsin Wong. Deep unsupervised pixelization. *ACM Transactions on Graphics*, 37(6): 243:1–243:11, 2018. SIGGRAPH Asia 2018. 3
- [10] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. *arXiv preprint arXiv:2207.12598*, 2022. A short version appeared at the NeurIPS 2021 Workshop on Deep Generative Models and Downstream Applications. 3
- [11] Susung Hong, Gyuseong Lee, Wooseok Jang, and Seungryong Kim. Improving sample quality of diffusion models using self-attention guidance. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 7462–7471, 2023. 3
- [12] Tero Karras, Miika Aittala, Tuomas Kynkäänniemi, Jaakko Lehtinen, Timo Aila, and Samuli Laine. Guiding a diffusion model with a bad version of itself. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. 3
- [13] Tuomas Kynkäänniemi, Miika Aittala, Tero Karras, Samuli Laine, Timo Aila, and Jaakko Lehtinen. Applying guidance in a limited interval improves sample and distribution quality in diffusion models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. 3

- [14] Lang Li. PixDiff-PIG: Palette-informed diffusion for pixel art generation. *Applied and Computational Engineering*, 211(1): 217–224, 2025. 3
- [15] Limbication. Pixel art LoRA for FLUX.2. <https://huggingface.co/Limbication/pixel-art-lora>, 2026. 3
- [16] nerijs. Pixel art XL. <https://huggingface.co/nerijs/pixel-art-xl>, 2023. LoRA for Stable Diffusion XL 1.0 (base), Hugging Face model repository. 3
- [17] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy V. Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Mahmoud Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Hervé Jegou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research (TMLR)*, 2024. 3
- [18] Gaurav Parmar, Richard Zhang, and Jun-Yan Zhu. On aliased resizing and surprising subtleties in GAN evaluation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11410–11420, 2022. 3
- [19] Seyedmorteza Sadat, Otmar Hilliges, and Romann M. Weber. Eliminating oversaturation and artifacts of high guidance scales in diffusion models. In *International Conference on Learning Representations (ICLR)*, 2025. 3
- [20] Seyedmorteza Sadat, Manuel Kansy, Otmar Hilliges, and Romann M. Weber. No training, no problem: Rethinking classifier-free guidance for diffusion models. In *International Conference on Learning Representations (ICLR)*, 2025. 3, 4
- [21] Seyedmorteza Sadat, Tobias Vontobel, Farnood Salehi, and Romann M. Weber. Guidance in the frequency domain enables high-fidelity sampling at low cfg scales. *arXiv preprint arXiv:2506.19713*, 2025. 3
- [22] George Stein, Jesse C. Cresswell, Rasa Hosseinzadeh, Yi Sui, Brendan Leigh Ross, Valentin Vilecroze, Zhaoyan Liu, Anthony L. Caterini, J. Eric T. Taylor, and Gabriel Loaiza-Ganem. Exposing flaws of generative model evaluation metrics and their unfair treatment of diffusion models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. 3
- [23] Singanallur V. Venkatakrishnan, Charles A. Bouman, and Brendt Wohlberg. Plug-and-play priors for model based reconstruction. In *IEEE Global Conference on Signal and Information Processing*, 2013. 3
- [24] Ryan Webster, Julien Rabin, Loic Simon, and Frederic Jurie. On the de-duplication of LAION-2B. In *arXiv preprint arXiv:2303.12733*, 2023. 3
- [25] Zongwei Wu, Liangyu Chai, Nanxuan Zhao, Bailin Deng, Yongtuo Liu, Qiang Wen, Junle Wang, and Shengfeng He. Make your own sprites: Aliasing-aware and cell-controllable pixelization. *ACM Transactions on Graphics*, 41(6):193:1–193:16, 2022. SIGGRAPH Asia 2022. 3
- [26] Shin-Ying Yeh. Detail-oriented pixelization based on contrast-aware outline expansion. <https://github.com/KohakuBlueleaf/Pixel10E>, 2024. Software; a classical (non-learned) image-processing pipeline. 3